

WA3854 Lab Guide

Intro to Web and HTML Fundamentals for Leaders



Part of
Accenture

© 2026 Ascendient, LLC

Revision 2.1.0 published on 2026-05-08

No part of this book may be reproduced or used in any form or by any electronic, mechanical, or other means, currently available or developed in the future, including photocopying, recording, digital scanning, or sharing, or in any information storage or retrieval system, without permission in writing from the publisher.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

To obtain authorization for any such activities (e.g., reprint rights, translation rights), to customize this book, or for other sales inquiries, please contact:

Ascendient, LLC

1 California Street Suite 2900

San Francisco, CA 94111

<https://www.ascendientlearning.com>

USA: 1-877-517-6540, email: getinfousa@ascendientlearning.com

Canada: 1-877-812-8887 toll free, email: getinfo@ascendientlearning.com

Lab Materials and Environment Guidelines

Ascendient Learning is pleased to provide lab materials and environments as part of your learning experience. To ensure a safe and effective learning experience for everyone:

● You may:

- Use the lab environment for your own coursework.
- Save any important files or data from the environment to your local computer or online storage (if allowed by your employer).
- Retain lab files for your personal use after the class ends.

● You may not:

- Share your lab materials or environment with anyone outside of your course.
- Store any personal, private, confidential, or proprietary company information within the lab environment.
- Rely on the lab environment for data storage, as all work is permanently deleted after each class.
- Misuse the lab environment for any illegal, malicious, or non-class-related activities.

🚨 Important

The lab environment is under Ascendient Learning's control and is not a part of your organization's IT ecosystem. Do not place your organization's information into the lab environment and always follow your organization's policies and procedures for working with IT systems.

Table of Contents

1. Evergreen Insurance Quote Page Add an Email Field.....	5
1.1. A Note About Keyboard Shortcuts.....	6
1.2. Why You're Doing This Lab	6
1.3. What's Already Set Up on Your Lab VM.....	7
1.4. Open the Folder from the Command Line	7
1.5. Run the App.....	8
1.6. Initialize Git for This Folder.....	10
1.7. Read the Existing Form	11
1.8. Add the Email Field.....	13
1.9. Try the Built-in Email Validation.....	15
1.10. Try the required Rule	16
1.11. See the Change in VS Code's Source Control Panel.....	17
1.12. See the Same Thing from the Command Line.....	18
1.13. Commit the Change.....	20
1.14. Review	20

Evergreen Insurance Quote Page Add an Email Field

LAB 1

In this lab, you will open the Evergreen Insurance Quote Page in Visual Studio Code (VS Code) and add a new **Email Address** field to the quote form. You'll watch the browser hot-reload as soon as you save, try out the browser's built-in email validation, and then see the change show up in VS Code's Source Control panel and from the `git diff` command.

You are not writing JavaScript this week. The change is one new HTML `<label>` plus one new `<input>` — about four lines of code. The goal is to see what an HTML change feels like end-to-end: edit, save, see it live, commit.

1.1. A Note About Keyboard Shortcuts

Your lab VM runs inside your browser. Some keyboard shortcuts (like `Ctrl+Shift+X`, `Ctrl+S`, `Ctrl+F`) may not always reach the VM — sometimes your browser catches them first. If a shortcut doesn't do anything, that's not you doing something wrong; just use the mouse/menu alternative that appears next to every shortcut in this lab. Every step in this lab lists both.

1.2. Why You're Doing This Lab

HTML forms are one of the most common places bugs live in real web applications. A missing `<label>`, a typo in an `id`, the wrong `type` attribute — any of these can silently break validation, accessibility, or submission. When completing this lab, you will know exactly what your engineers mean when they say "let me add that email field to the intake form" — and roughly what can go wrong.

This lab reuses the full flow from Week 1 (open project → run with Live Server → edit → save → see update → check Git) and adds one new thing: the edit itself is a real HTML element, not a one-word text change.

1.3. What's Already Set Up on Your Lab VM

Your lab machine runs Ubuntu Linux and comes pre-installed with everything you need for this lab. You do not need to install anything.

- **Visual Studio Code** — the code editor you used in Week 1.
- **Live Server extension** — you installed this in Week 1; it is still installed.
- **Git** — initialized in the Week 1 lab; this week's lab folder has its own fresh Git setup.
- **Firefox** and **Chrome** — either browser can be used to view the Evergreen quote page.

1.4. Open the Folder from the Command Line

You'll navigate to the lab folder using the terminal, then open it in VS Code from the command line.

1. Open a terminal. Three ways — use whichever feels easiest:
 - **From the dock:** Click the **Terminal** icon in the dock at the bottom of the screen (the black icon with `>_`).
 - **From Activities:** Click **Activities** in the top-left corner, type `Terminal` in the search box, then click the Terminal icon in the results.
 - **With a keyboard shortcut:** Press `Ctrl+Alt+T`.
2. Navigate into the lab folder:

```
cd ~/Labfiles/evergreen-quote-add-email-field
```

Note

If you get "No such file or directory," run `ls ~/Labfiles` to see what's actually there. The `~` symbol is shorthand for your home folder.

3. Confirm you're in the right place:

```
ls
```

You should see `index.html`, `styles.css`, `app.js`, and `README.md`.

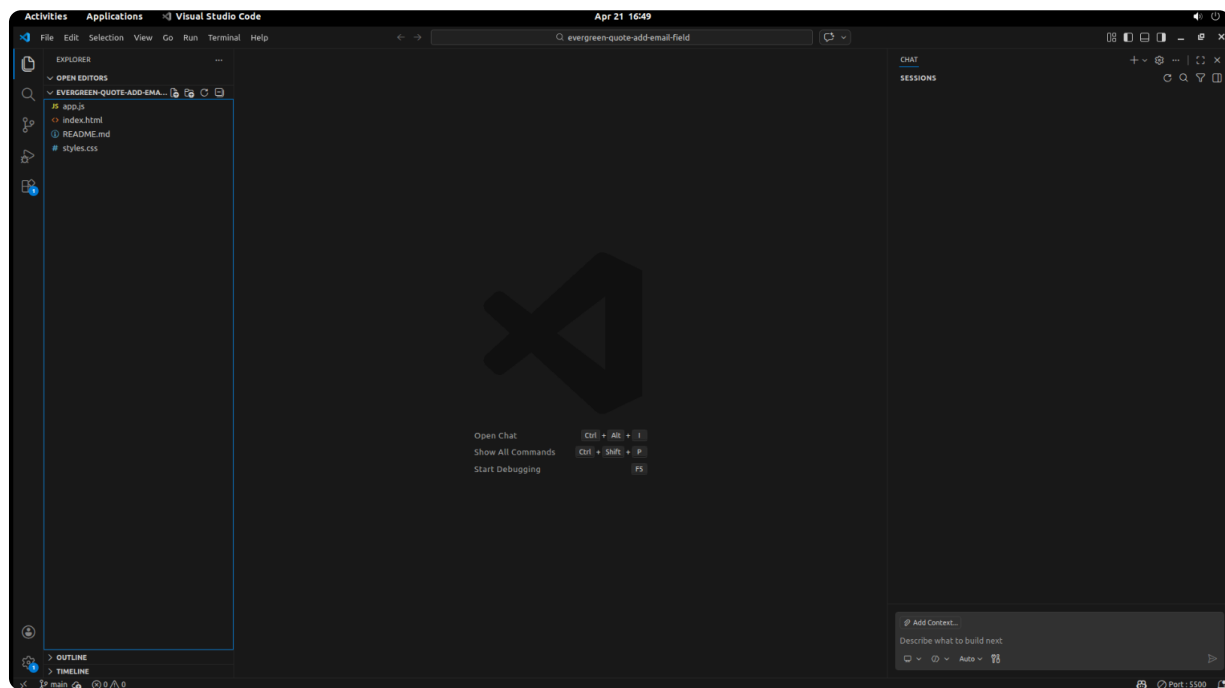
4. Open the folder in VS Code from the terminal:

```
code .
```

The `.` means "the current folder." VS Code will open with this folder loaded in its sidebar.

Note

If VS Code asks "**Do you trust the authors of the files in this folder?**", click **Yes, I trust the authors**. The Evergreen files are provided by your course.



1.5. Run the App

1. Open the Explorer panel if it is not already visible. Two ways:
 - **With the mouse:** Click the top icon in the Activity Bar (two overlapping pages).

- **With a keyboard shortcut:** Press `Ctrl+Shift+E`.
2. Right-click on `index.html` in the Explorer and choose **Open with Live Server**.
 3. The Evergreen quote page should open in your browser (Firefox or Chrome — either works). It should look identical to the one you saw in Week 2: dark-green header, "Coverage that grows with you" hero, and a quote form with Full Name, ZIP Code, Coverage Type, and Coverage Amount fields.

4. Try filling it out: enter a name, a ZIP code, pick a coverage type, drag the slider, and click **Calculate Premium**. An estimated monthly premium should appear below the form. Notice what the form does **not** ask for: an email address. That is what you are about to add.
5. Leave the browser tab open. Switch back to VS Code.

1.6. Initialize Git for This Folder

Just like in Week 1, you need to tell Git to start tracking this folder before you can see changes.

1. Open the Source Control panel. Two ways:
 - **With the mouse:** Click the third icon down in the Activity Bar (a branching line with dots).
 - **With a keyboard shortcut:** Press `Ctrl+Shift+G`.
2. You should see the **Initialize Repository** button. Click it.
3. Make an initial commit so there is a baseline to compare future changes against:
 - Click inside the **Message** box at the top of the Source Control panel.
 - Type `Week 3 starting state`.
 - Click the blue **Commit** button. If VS Code asks *"Would you like to stage all your changes and commit them directly?"*, click **Yes**.
4. The Source Control panel should now be empty under **Changes** — everything has been committed. Your project has exactly one commit: the starting state.

Why this step: Starting every week's lab with a fresh `Initial commit` gives you a clean baseline to compare your changes against. Engineers do this once at the beginning of a project, not every week, but for teaching purposes it lets you see the entire Week 3 change as a single diff at the end.

Note

The **Publish Branch** button that appears after you commit is for sharing your code on GitHub. You do not need to click it for this lab.

1.7. Read the Existing Form

Before you add anything, you need to know what's already there. Think of this like reading a recipe before you start chopping ingredients.

1. In the Explorer panel, click `index.html` to open it in the editor.
2. Find the `<form>` element in `index.html`. Two ways:
 - **With the mouse:** Scroll down until you see `<form id="quote-form" class="quote-form">` — it is around line 28.
 - **With a keyboard shortcut:** Press `Ctrl+F` to open the Find box, type `<form`, and press `Enter`.
3. Look at the very first `<label>` inside the form:

```
<label>
  Full Name
  <input type="text" id="fullName" placeholder="Jane Doe"
required />
</label>
```

Read this carefully. The `<label>` wraps two things:

- The text **"Full Name"** — what the user sees next to the field
- An `<input>` element with:
 - `type="text"` — a plain text box
 - `id="fullName"` — a unique name used by the JavaScript in `app.js`
 - `placeholder="Jane Doe"` — the gray hint text shown when the field is empty
 - `required` — the form will not submit unless this field has a value

Why this matters: Every field in a well-built form looks approximately like this. Once you can read one, you can read them all. The email field you are about to add will follow the same pattern, changed in three places: the label text, the input `type`, and the input `id`.

4. Scroll through the rest of the form. Identify the `<label>` + `<input>` pair for **ZIP Code**, the `<label>` + `<select>` for **Coverage Type**, and the `<label>` + `<input type="range">` for **Coverage Amount**. Notice all of them use the same wrapping pattern.

```
index.html X
index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8" />
5    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6    <title>Evergreen Insurance – Get a Quote</title>
7    <link rel="stylesheet" href="styles.css" />
8  </head>
9  <body>
10   <header class="site-header">
11     <div class="brand">Evergreen Insurance</div>
12     <nav class="site-nav">
13       <a href="#">Home</a>
14       <a href="#">Coverage</a>
15       <a href="#">Claims</a>
16       <a href="#">Contact</a>
17     </nav>
18   </header>
19
20   <main>
21     <section class="hero">
22       <h1>Coverage that grows with you</h1>
23       <p class="tagline">Get a personalized quote in under a minute.</p>
24     </section>
25
26     <section class="quote-form-section">
27       <h2>Get Your Quote</h2>
28       <form id="quote-form" class="quote-form">
29         <label>
30           Full Name
31           <input type="text" id="fullName" placeholder="Jane Doe" required />
32         </label>
33
34         <label>
35           ZIP Code
36           <input type="text" id="zip" placeholder="02101" maxlength="5" required />
37         </label>
38
39         <label>
40           Coverage Type
41           <select id="coverageType" required>
42             <option value="">Select coverage</option>
43             <option value="auto">Auto</option>
44             <option value="home">Home</option>
45             <option value="renters">Renters</option>
46           </select>
47         </label>
48
49         <label>
```

1.8. Add the Email Field

You are going to add a new `<label>` and `<input>` directly below the **Full Name** field.

1. In `index.html`, find the closing `</label>` tag right after the Full Name input. The four lines look like this:

```
<label>
  Full Name
  <input type="text" id="fullName" placeholder="Jane Doe"
  required />
</label>
```

2. Place your cursor at the end of that closing `</label>` line and press `Enter` twice to make a blank line, then type the following exactly:

```
<label>
  Email Address
  <input type="email" id="email" placeholder="jane@example.com"
  required />
</label>
```

Tip

VS Code may offer auto-complete suggestions as you type — you can accept them with `Tab`, or press `Escape` to dismiss them and keep typing by hand.

3. Double-check your work. The form section in `index.html` should now read:

```
<form id="quote-form" class="quote-form">
  <label>
    Full Name
    <input type="text" id="fullName" placeholder="Jane Doe"
required />
  </label>

  <label>
    Email Address
    <input type="email" id="email"
placeholder="jane@example.com" required />
  </label>

  <label>
    ZIP Code
    <input type="text" id="zip" placeholder="02101"
maxlength="5" required />
  </label>
  ...

```

What you just did: You added an HTML element. `type="email"` tells the browser to expect a valid email address format (like `something@something.something`) and to reject anything else when the form is submitted. `required` tells the browser the field must not be empty. `id="email"` is a unique handle the JavaScript in `app.js` could use to read the value (not yet — that comes in JavaScript week).

4. Save the file. Two ways:

- **With the mouse:** Click the **File** menu at the top of the VS Code window, then click **Save**.
- **With a keyboard shortcut:** Press `Ctrl+S`.

5. Switch to your browser tab. The page should update automatically — the new **Email Address** field should now appear between **Full Name** and **ZIP Code**.

Evergreen Insurance

Home Coverage Claims Contact

Coverage that grows with you

Get a personalized quote in under a minute.

Get Your Quote

Full Name
Jane Doe

Email Address
jane@example.com

ZIP Code
02101

Coverage Type
Select coverage

Coverage Amount: \$50000

Calculate Premium

© 2025 Evergreen Insurance. For training purposes only.

Why this matters: This is the inner loop of front-end work — type HTML, save, see the browser update. Your engineers do this hundreds of times a day. The feedback is this fast because Live Server is watching the file for changes.

1.9. Try the Built-in Email Validation

You did not write a single line of validation code, but the browser is already validating the email field for you. Verify this by submitting the form with a bad value.

1. In your browser, fill in the form:
 - Full Name: your name
 - Email Address: type `abc` (not a valid email)
 - ZIP Code: `02101`
 - Coverage Type: pick one
 - Coverage Amount: leave the slider wherever it is
2. Click **Calculate Premium**.

3. The browser should pop up a tooltip on the Email Address field saying something like **"Please include an '@' in the email address."** The form does not submit.

Evergreen Insurance

Home Coverage Claims Contact

Coverage that grows with you
Get a personalized quote in under a minute.

Get Your Quote

Full Name
Jane Doe

Email Address
abc

ZIP
12101

Please include an '@' in the email address. 'abc' is missing an '@'.

Coverage Type
Auto

Coverage Amount: \$50,000

Calculate Premium

© 2025 Evergreen Insurance. For training purposes only.

Why this matters: Every modern browser ships HTML's built-in validation. `type="email"` gets you format checking. `required` gets you "don't leave this blank." `maxlength` gets you length limits. Engineers reach for custom JavaScript validation only when the built-in rules aren't enough — and they should always start from these defaults.

4. Now try with a valid email:
 - Change the Email Address to something like `you@example.com`
 - Click **Calculate Premium** again
5. The premium box should now appear below the form, showing an estimated monthly premium. The form submitted successfully because every required field has a valid value.

1.10. Try the `required` Rule

1. Clear the Email Address field so it is empty.
2. Click **Calculate Premium**.

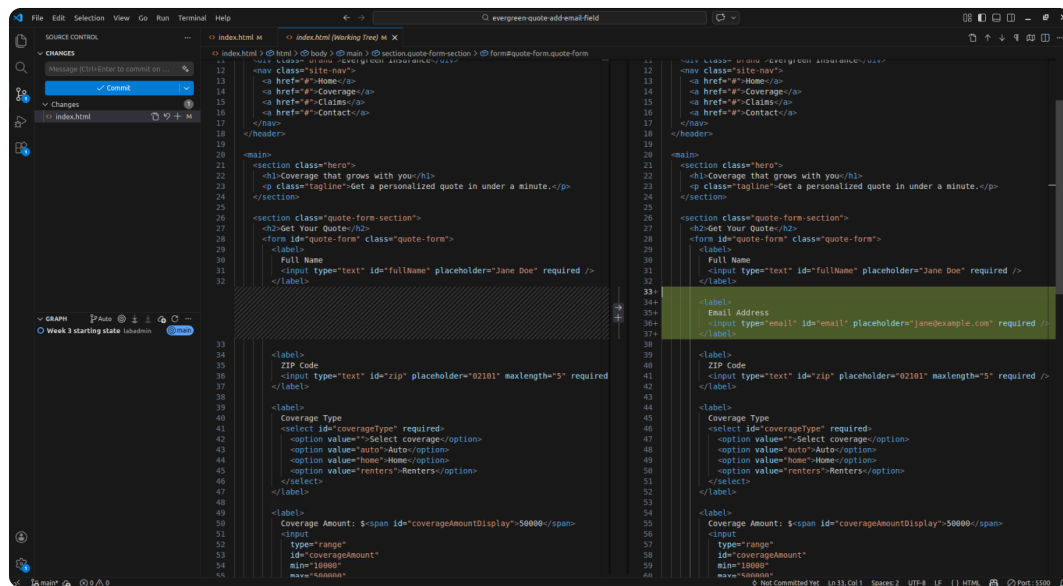
3. The browser should pop up a tooltip on the Email Address field saying something like **"Please fill out this field."** The form does not submit.

Why this matters: `required` gave you the "don't submit this form until the user fills in the field" behavior you would otherwise need JavaScript to build.

4. Put a valid email back in the field and submit successfully once more. Leave the page as-is and switch back to VS Code.

1.11. See the Change in VS Code's Source Control Panel

1. Open the Source Control panel. Two ways:
 - **With the mouse:** Click the third icon down in the Activity Bar.
 - **With a keyboard shortcut:** Press `Ctrl+Shift+G`.
2. You should see `index.html` listed under **Changes** with an `M` next to it. `M` means "modified."
3. Click on `index.html` in the Source Control panel. VS Code will open a **diff view**:
 - The left pane shows the file as it was at your last commit (the Week 3 starting state).
 - The right pane shows the file as it is now, with your new `<label>` highlighted in green.



4. Close the diff view by clicking the **X** on its tab at the top of the editor.

Why this matters: This diff view is exactly what your engineers look at when they are about to open a pull request. When they say "let me just sanity-check my changes before I push," this is the view they are sanity-checking. Knowing what a clean, small, purposeful diff looks like will make every PR review you attend easier to follow.

1.12. See the Same Thing from the Command Line

You have just seen your change through VS Code's graphical panel. Now see it through the terminal.

1. Open VS Code's integrated terminal. Two ways:
 - **With the mouse:** Click the **Terminal** menu at the top of the VS Code window, then click **New Terminal**.
 - **With a keyboard shortcut:** Press **Ctrl+`**.
2. The terminal opens inside your project folder. Run:

```
git status
```

You should see `modified: index.html` — the same information the Source Control panel showed you.

3. Now run:

```
git diff
```

You'll see the same before/after change — in text form, with red `-` marks on removed lines and green `+` marks on added lines. Because you only added lines (you didn't remove anything), every changed line should be green.

```
• labadmin@ip-172-16-141-178:~/Labfiles/evergreen-quote-add-email-field$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
• labadmin@ip-172-16-141-178:~/Labfiles/evergreen-quote-add-email-field$ git diff
diff --git a/index.html b/index.html
index dal8e39..646a77b 100644
--- a/index.html
+++ b/index.html
@@ -31,6 +31,11 @@
     <input type="text" id="fullName" placeholder="Jane Doe" required />
   </label>

+   <label>
+     Email Address
+     <input type="email" id="email" placeholder="jane@example.com" required />
+   </label>
+
   <label>
     ZIP Code
     <input type="text" id="zip" placeholder="02101" maxlength="5" required />
• labadmin@ip-172-16-141-178:~/Labfiles/evergreen-quote-add-email-field$
```

Why this matters: Graphical and command-line Git views are the same thing, presented differently. You don't need to pick a favorite — you just need to recognize both.

1.13. Commit the Change

You've verified the change works in the browser, in VS Code, and from the command line. Now record it in Git as a permanent part of the project's history.

1. In the Source Control panel, click inside the **Message** box and type:

```
Add required email field to quote form
```

Why the message matters: A good commit message describes **what** changed and, when it's not obvious, **why**. "Added stuff" is a bad commit message. "Add required email field to quote form" is a good one — future-you can read it three months later and know exactly what this commit did without opening the diff.

2. Click the blue **Commit** button. If VS Code asks to stage all your changes, click **Yes**.
3. Run `git status` in the terminal one more time. It should say `nothing to commit, working tree clean`.
4. Run `git log --oneline` in the terminal:

```
git log --oneline
```

You should see two commits — your **Week 3 starting state** and your new **Add required email field to quote form** — with the most recent one on top.

Why this matters: `git log` is the engineering version of a changelog. Engineers read it constantly to figure out who changed what and when. Compact commits like yours make this log useful; bloated commits that touch fifty files with a vague message make it useless.

1.14. Review

In this lab, you:

- Opened an existing HTML project in VS Code from the command line.
- Read a `<form>` block and identified its `<label>` + `<input>` pattern.

- Added a new `<label>` and `<input type="email" required />` in the correct place.
- Watched the browser hot-reload your change.
- Saw HTML's built-in `type="email"` validation in action.
- Saw HTML's built-in `required` validation in action.
- Compared your file against the starting state using both VS Code's Source Control panel and `git diff`.
- Committed the change with a useful message.

You wrote four lines of HTML. You watched those four lines turn into a working, validated form field. That is what "adding a field to the quote form" actually looks like — and now you've lived it.

Next week you'll revisit this page in CSS week to restyle the premium box. The email field you just added will still be there.